

Introduction to LLVM with Compiler basics

Compiler Engineering #1

Yuri Takigawa

2026/06/28

M1, Taura Laboratory, Creative Information, Graduate School of Information Science Technology, The University of Tokyo

Contents

1. LLVM setup 2

2. Compiler recap 4

3. Building LLVM 12

1. LLVM setup

Installation

1. Glance at <https://releases.llvm.org/>
2. Execute followings replacing `INSTALL_PREFIX` with the path where you want the tools to be installed.

```
$ git clone https://github.com/PacktPublishing/LLVM-Code-Generation.git
$ cd LLVM-Code-Generation/ch1
$ bash setup_env.sh ${INSTALL_PREFIX}
```

2. Compiler recap

Compiler Components

When building a C file, Clang acts as a *driver* for a series of tools.

- invokes the frontend (Clang project in LLVM)
- passes down the result to the backend (LLVM backend)
 - that produces an object file
- those object file gets linked with the standard library (the `libc` project in LLVM) by the linker (the `lld` project in LLVM)

To build a properly functioning compiler, you will need to build at least **linker** and **standard library**.

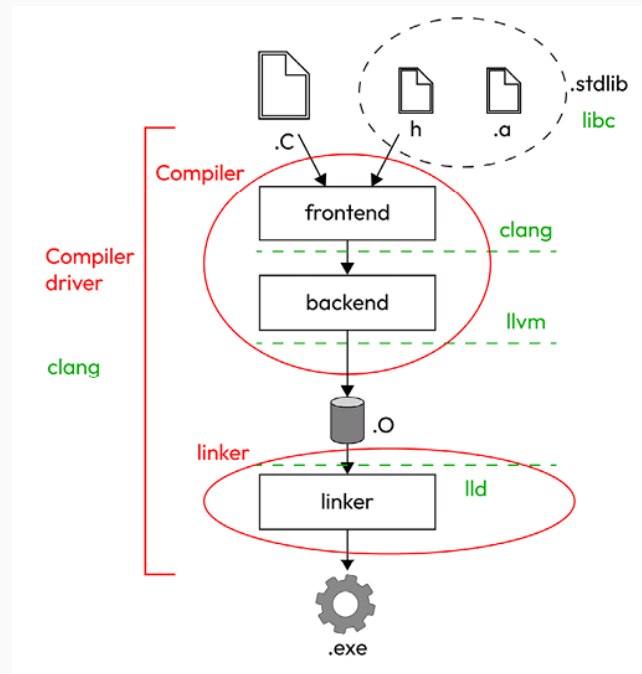


Figure 1: The different components of a compiler ©Figure1.2 of the book

Building Clang to grasp the compiler tool chain

Building Clang (projects) always includes backend.

Assuming

- `LLVM_SRC` is the path where you want to have the LLVM source code
- `CLANG_BUILD` is the path where you want the build of Clang to happen

please run the following:

```
$ git clone https://github.com/llvm/llvm/project.git ${LLVM_SRC}
$ mkdir -p ${CLANG_BUILD}
$ cd ${CLANG_BUILD}
$ cmake -DLLVM_ENABLE_PROJECTS=clang -GNinja -DCMAKE_BUILD_TYPE=Release ${LLVM_SRC}/llvm
$ ninja clang
```

Experimenting with Clang: Frontend

Frontend validates that the input file is syntactically and semantically correct and produces the LLVM IR.

- **Preprocessor** expands macros (e.g., `#include`)
- **Sema** validates the syntax and semantics of the program.
- **Codegen** produces the LLVM IR.

To stop	Command
After the preprocessor	<code>clang -E</code>
After syntax checking	<code>clang -fsyntax-only</code>
After LLVM IR code generation	<code>clang -O0 -emit-llvm -S</code>

Experimenting with Clang: Backend & Assembler

Backend translates the LLVM IR to target specific instructions.

- **Middle-end optimization**: LLVM IR to LLVM IR optimizations.
- **Assembly generation**: Target-specific IR to assembly code.

Assembler translates assembly code to an object file.

To stop	Command
After the middle-end optimizaiton	<code>clang -O<1 2 3 s z> -emit-llvm -S</code>
After assembly generation	<code>clang -S</code>
After the assembler	<code>clang -c</code>

Understanding compiler jargon¹

- **Target** is the hardware architecture that a program will run on. “*Targetting* of instruction selection” means that we will modify the instruction selection transformation so that it supports a *specific target*.
- **Host** is the device that runs the compiler. In *cross-compilation*, you can run a compiler that produces code for an AArch64 (used in iPhone) on an x86 host (windows laptop).
- **Lowering** is the notion that as your input program is being compiled, it goes through various stages that progressively lower its level of abstraction all the way down to the final assembly of the target machine.
- **Canonical form** is about the recommended way of representing something, which is ultimately useful to make an agreement across different stages of compilation. Let’s learn by example of expressions $a = b + 2$ and $a = 2 + b$:
 - computes the same results but offer different ways to represent their computations.
 - For instance, a rule could be to put the constants on the right-hand side of binops.
 - LLVM offers APIs to canonicalize your IR.

Time

- **Build time**: The time it takes to build the compiler; for example, the time it takes for Ninja to complete the build of LLVM.
- **Comile time**: The time it takes for the compiler to process a file; for example the time it takes for Clang to produce an object file from the source file.
- **Runtime**: The time it takes for the final binary to execute; for example, the time it takes to run the binary produced by Clang.

Understanding compiler jargon

Application binary interface (ABI)

ABI formalize how the handshake happens between the caller function and the callee function at a low level.

- Specifically, ABI defines how and where the *arguments* of a function are set on the caller side so that they can be retrieved on the callee side.
- the ABI can harm the whole compiler stack because it affects the function signature, which is typically determined by the frontend, all the way down to the backend, which must set the right stack alignment, reserve dedicated registers, and so on.
- As long as compilers follow the rules of the ABI, a function compiled with a compiler can talk to a function compiled with a different compiler.

If you are writing your own backend, you must define your own ABI, but should use an existing one and derive yours from that.

3. Building LLVM

Configuring the build system

Official build system is CMake.

CMake variables

CMake comes with some built-in variables that can be used to customize some key aspects of the build process, which are recognized with their name starting with **CMAKE_**.

CMake command-line options

CMake also supports command-line options, but for all intent and purposes, we will mention only three here

- **-D<var>=<value>**: This defines the value of a CMake variable.
- **-G<generatorName>**: This generates a build system for the specified generator.
- **-C<pathToCacheFile>**: This preloads a cache file that pre-set some CMake variables¹

¹cache files are useful for sharing specific configurations and avoiding setting all the variables manually.

Configuring the build system

Simplest commands you can run from your build directory to configure the LLVM's build system:

```
$ cmake -GNinja -DCMAKE_BUILD_TYPE=Debug ${LLVM_SRC}/llvm
```

Commonly used CMake variables

Variable	Value	Meaning
Standard options		
CMAKE_BUILD_TYPE	Debug	Build for a smooth debug experience: Assertions: Enabled Optimizations: Disabled Debug info: Enabled Produces a large and slow compiler.
	Release	Build an optimized compiler: Assertions: Disabled Optimizations: Enabled Debug info: Disabled Produces a smaller and faster compiler.
CMAKE_C_COMPILER	<path>	Specify the path to the C compiler. This is particularly useful when bootstrapping or cross-compiling the compiler. We will not cover these topics, but at least you know where to look if you are interested in this.
CMAKE_CXX_COMPILER	<path>	Specify the path to the C++ compiler.
CMAKE_INSTALL_PREFIX	<path>	Specify where to install the final artifacts.
Faster build time		
LLVM_TARGETS_TO_BUILD	Target1;...	Specify the list of backends to build (semicolon separated). Target1 , and so on, must match the directory name of one of the backends in <code>\${LLVM_SRC}/llvm/lib/Target</code> . Default to the <code>all</code> special value, which builds all the ~20 non-experimental LLVM backends.
LLVM_OPTIMIZED_TABLEGEN	BOOL	Specify whether or not to build TableGen in optimized mode. We will cover TableGen in more detail in the dedicated chapter but the gist of it is unless you are developing a TableGen backend, you will likely want to set this variable to speed up your build.
Notably useful		
BUILD_SHARED_LIBS	BOOL	Build libraries as shared libraries. This avoids the link steps for the different executables, but this means they are not self-contained anymore and you have to “ship” the shared libraries alongside them. For local development, this may be worth it, although the debug experience may not be that great.
LLVM_ENABLE_ASSERTIONS	BOOL	Enable or disable assertions. Using this option, you can for instance enable the assertions in a release build, which can be useful to diagnose some issues while not paying the price of a full debug build.
LLVM_ENABLE_PROJECTS	Project1;...	Build Project1 , and so on, on top of the LLVM core. The project names must match the directory names at the root of <code>\${LLVM_SRC}</code> (e.g., <code>clang;mlir</code> will build both Clang and MLIR with the rest of LLVM).

Here is recommended one for a faster build time that still features a debuggable compiler:

```
$ cmake -GNinja -DCMAKE_BUILD_TYPE=Debug -DLLVM_TARGETS_TO_BUILD="X86;AArch64" -  
DLLVM_OPTIMIZED_TABLEGEN=1 ${LLVM_SRC}/llvm
```


Ninja is a tool that drives a build system, especially making sure that artifacts are built following the order of the dependencies described in this build system.

Building the core LLVM project and try a series of things

To build LLVM, RUN

```
$ ninja
```

Then, follow the instructions for **testing a compiler**

Understanding the directory structure

From the highest level of repository,

- LLVM code base is organized into projects: Clang, MLIR, the **LLVM debugger (LLDB)**, etc.
- You need to use these directory names (like `clang`, `mlir`, `lldb` and so on) in the `LLVM_ENABLE_PROJECTS` CMake variable to build the related projects.

The core of LLVM `llvm` is primarily composed of several interesting pieces

- `include`: public headers
- `lib`: different libraries
- `tools`: developer-facing tools
- `unittests` and `test`: tests for all the tools and libraries
- `utils`: various utility tools such as `FileCheck` and `llvm-lit`

When you implement something, follow the unwritten “rule”

List the content of the `llvm/lib/` directory, then see `llvm/include/llvm/` and

Overview of the LLVM components

Generic LLVM goodness

LLVM provides a lot of well-optimized data structures and utilities that can be reused as is.

- In **ADT** directory, you can find *containers* like link lists, maps etc., as well as portable implementations for *arbitrary-sized integer types*.
- In **Support**, wrappers for string and file system manipulations, error handling etc.

Working with the LLVM IR

LLVM IR is the main *exchange format* for everything built around LLVM. It lives in the IR directories (both **include/llvm/IR**, and **lib/IR**).

- Optimizations transforming/analyzing the IR live in **Analysis** and **Transforms**.
- The binary and textual representation of the IR is handled in **Bitcode**, **IRReader**, and **IRWriter**.

Generic backend infrastructure